Instituto Tecnológico de Costa Rica

Escuela de Ingeniería Electrónica
Laboratorio de Control Automático

Carrito Evasor de Obstáculos

Proyecto Fase 2

Estudiantes:

Nicole Corrales, Kimberley Morales, Angie Hernández

Profesor: Pablo César Rodríguez Vargas

Fecha: 20 de Noviembre 2025

Contents

1 Introducción **2**

2 Proceso de Construcción **3**

3 Modelo Empírico de la Planta **3**

4 Diseño del Controlador **6**

5 Resultados **7**

5.1 Sistema sin compensación 7

5.2 Sistema compensado con PID 8

6 Análisis de Resultados **9**

7 Conclusiones y Recomendaciones **9**

7.1 Conclusiones 9

7.2 Recomendaciones 10

8 Bibliografía **10**

A Código Fuente **11**

B Planos de Construcción **22**

Abstract

En este proyecto se desarrolla una planta constituida por un carro móvil evasor de obstáculos, cuya dinámica se modela a partir de datos experimentales con el fin de diseñar un controlador adecuado, el sistema emplea un sensor ultrasónico para medir la distancia frente a los obstáculos, dos motores DC controlados mediante un driver L298N, una rueda loca frontal y un módulo Arduino como unidad de procesamiento. A partir del modelo empírico obtenido, se implementa un controlador PID orientado a mejorar la estabilidad y el comportamiento dinámico del vehículo ante variaciones en su entorno.

Los resultados experimentales demuestran que el controlador incrementa la estabilidad del sistema y permite un desplazamiento más preciso y seguro, manteniendo una distancia adecuada frente a obstáculos y mejorando la respuesta general del carro.

Palabras clave: control PID, modelo empírico, carro evasor de obstáculos, control automático, dinámica de sistemas.

1 Introducción

El proyecto desarrollado consiste en el diseño, modelado e implementación de un sistema de control aplicado a un carro móvil evasor de obstáculos, la motivación radica en llevar a la práctica los principios teóricos estudiados en el curso de Control Automático, aplicando metodologías reales de modelado, identificación, diseño de controladores y validación experimental. El trabajo permite integrar el análisis matemático con experimentación física, el uso de herramientas como MATLAB y la puesta en marcha de un sistema funcional construido desde cero.

La planta está conformada por un vehículo móvil de dos ruedas motrices y una rueda loca frontal, equipado con un sensor ultrasónico encargado de medir la distancia frontal d_f ante un obstáculo, el sistema utiliza un Arduino como unidad de procesamiento, un driver L298N para accionar los motores y una señal PWM compartida como variable manipulada. De esta forma, la velocidad lineal del carro depende directamente del valor de la señal de control aplicada, permitiendo influir sobre su capacidad de aproximarse o alejarse de objetos en su trayectoria.

Con base en los datos experimentales obtenidos, se construyó un modelo empírico que describe la relación dinámica entre la señal PWM aplicada a los motores (entrada u) y la distancia frontal medida (salida $y = d_f$). Este modelo permitió diseñar un controlador PID orientado a regular la distancia del vehículo respecto a un obstáculo, manteniéndola en un valor deseado d_{ref}. El objetivo de control consiste en garantizar estabilidad, rechazo de perturbaciones y una respuesta suave ante variaciones inesperadas en el entorno.

Este documento presenta el desarrollo completo del proyecto, desde la construcción física del prototipo hasta el diseño, simulación e implementación del controlador, mostrando el desempeño obtenido con y sin compensación.

2 Proceso de Construcción

El prototipo del carro evasor de obstáculos fue construido mediante un diseño modular fabricado completamente en impresión 3D, cada componente estructural se elaboró como una pieza independiente para facilitar el ensamblaje, mantenimiento y reemplazo, se desarrollaron módulos específicos para la base principal, soporte de baterías, soporte del driver L298N, plataforma frontal, bases de motor y la estructura destinada al sensor ultrasónico y al servomotor.

La locomoción del vehículo se realiza mediante dos motores DC laterales, cada uno fijado a una base impresa que permite una sujeción rígida y estable, en el extremo frontal se incorpora una rueda loca montada dentro de una cavidad diseñada para garantizar un movimiento suave y evitar atascos al cambiar de dirección. La estructura general proporciona rigidez mecánica y distribuye el peso de forma uniforme sobre el chasis.

Para la detección de obstáculos se empleó un sensor ultrasónico HC-SR04, montado en la parte frontal sobre una base elevada creada para asegurar que el sensor permanezca perpendicular a los objetos en su trayectoria. El soporte fue fijado mediante tornillería, logrando un alineamiento estable y evitando vibraciones, inicialmente, los motores fueron instalados en posición vertical, pero debido a problemas de fragilidad en la base impresa, se rediseñó la instalación para colocarlos en posición horizontal, mejorando así la resistencia mecánica del conjunto.

El sistema se alimenta mediante dos baterías recargables de 9 V conectadas en serie para incrementar el voltaje disponible, estas alimentan directamente el driver L298N, el cual distribuye energía tanto a los motores como al Arduino. No fue necesario el uso de una protoboard, ya que todas las conexiones funcionales se realizaron a través de los bornes del driver y los pines del microcontrolador, reduciendo el cableado y mejorando la estabilidad eléctrica.

Finalmente, se integró todo el sistema dentro del chasis 3D, asegurando accesibilidad al Arduino y al driver, espacio para las baterías y buena ventilación alrededor de los componentes.

3 Modelo Empírico de la Planta

Para obtener un modelo dinámico que describiera el comportamiento del carro móvil frente a variaciones de la señal de control, se realizaron tres experimentos utilizando diferentes niveles de PWM aplicados a los motores: 90, 110 y 130. En cada prueba, el vehículo se desplazó directamente hacia un obstáculo mientras se registraban simultáneamente la señal de entrada $u(t)$ (PWM) y la distancia frontal $d_f(t)$ medida mediante el sensor ultrasónico HC-SR04. Las señales se adquirieron con un tiempo de muestreo de $T_s = 0.05\,\mathrm{s}$.

Previo a cada experimento, se añadió un bloque de muestras en reposo para garantizar un inicio consistente y permitir que el algoritmo de identificación considerara correctamente el instante del escalón. Los datos fueron organizados en objetos `iddata` de

MATLAB para su posterior procesamiento.

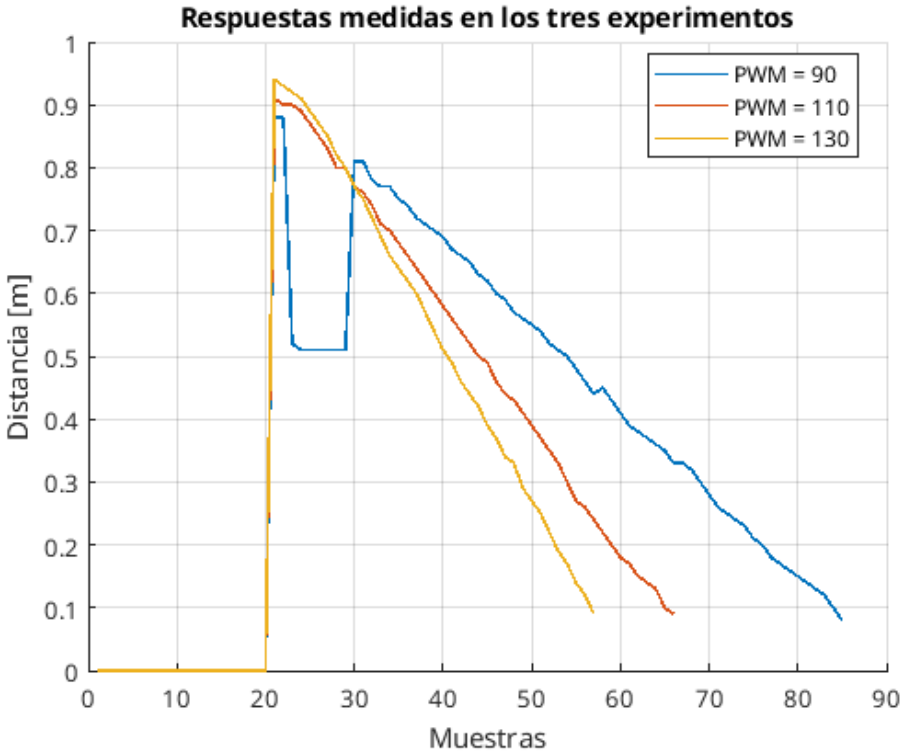


Figure 1: Respuestas medidas en los tres experimentos realizados con PWM = 90, 110 y 130.

La Fig. 1 muestra las respuestas medidas a los distintos niveles de PWM, se observa que valores mayores de PWM producen un acercamiento más rápido al obstáculo, lo cual confirma la naturaleza monótona del sistema dinámico.

Del conjunto de datos disponible, se seleccionó la medición asociada a PWM = 110 como referencia para la identificación del modelo, por presentar un comportamiento intermedio y menos afectado por saturación o ruido. A partir de estos datos se estimaron distintos modelos mediante el comando `tfest`, evaluando las siguientes estructuras:

- Modelo de un polo (primer orden)
- Modelo de dos polos
- Modelo de un polo con retardo puro

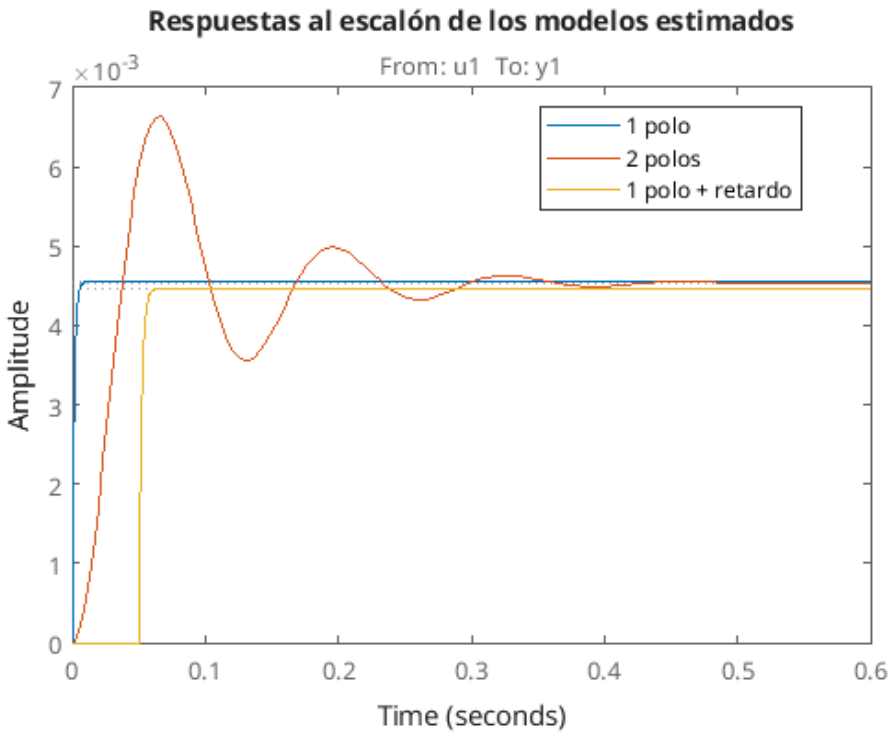


Figure 2: Comparación de la respuesta al escalón de los modelos estimados.

La Fig. 2 muestra la comparación entre los distintos modelos estimados. El modelo de dos polos presentó el mejor ajuste global, logrando reproducir con mayor fidelidad la curvatura inicial y el comportamiento transitorio observado en los datos experimentales. A diferencia del modelo de un polo, que resulta más simplificado, el modelo de dos polos captura mejor los efectos dinámicos adicionales presentes en la señal real, reduciendo el error entre el modelo y la medición.

El modelo seleccionado corresponde entonces a un sistema de dos polos, cuya estructura general es:

$$G(s) = \frac{K}{(s+p_1)(s+p_2)},$$

con los parámetros estimados mediante `tfest`:

- Ganancia: $K \approx 4.2 \times 10^{-3}$
- Primer polo: $p_1 \approx 20.8$
- Segundo polo: $p_2 \approx 1.64$

Este modelo describe de mejor manera la dinámica del sistema, especialmente en el régimen transitorio, y constituye la base utilizada para el diseño del controlador PID implementado.

Finalmente, la respuesta al escalón en lazo cerrado con el controlador PID diseñado (ver Sección correspondiente) se muestra en la Fig. 3, la cual evidencia que el modelo identificado permite obtener una compensación estable y con buen desempeño dinámico.

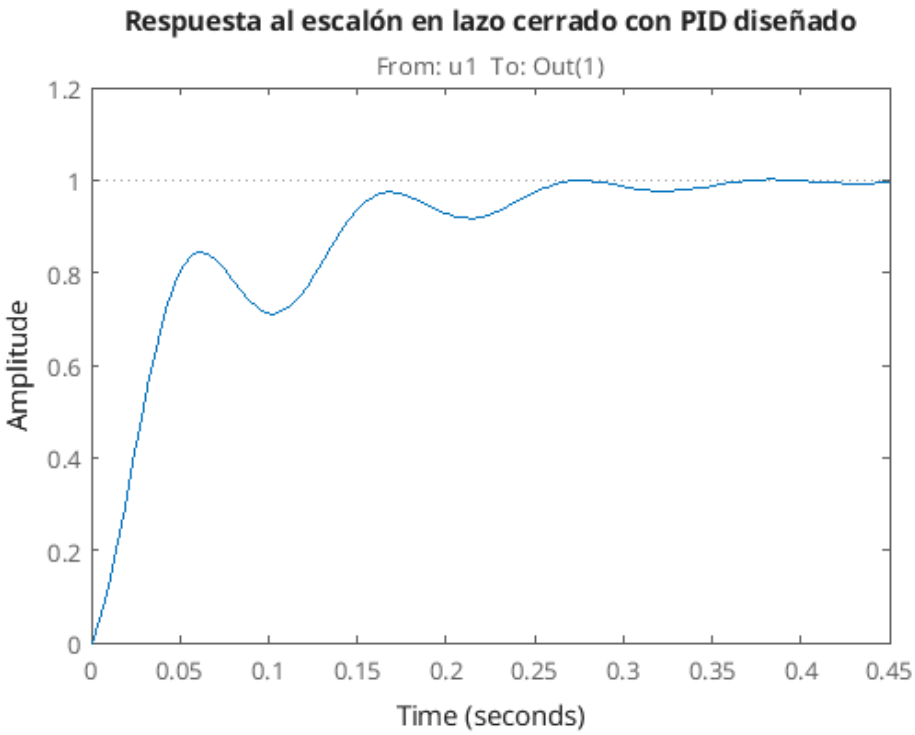


Figure 3: Respuesta al escalón en lazo cerrado utilizando el PID diseñado.

4 Diseño del Controlador

El diseño del controlador se realizó utilizando el modelo empírico identificado previamente para la planta, a partir de los datos experimentales, el sistema se aproximó mediante un modelo discreto de dos polos con tiempo de muestreo $T_s = 0.05\,\mathrm{s}$, el cual fue empleado dentro de MATLAB para el diseño y ajuste de un controlador PID orientado a regular la distancia frontal d_f a un valor de referencia d_{ref}.

El objetivo principal del controlador era reducir el error en estado estacionario y mejorar la estabilidad del acercamiento al obstáculo, evitando oscilaciones o convergencias bruscas. Para ello, se utilizó la herramienta `pidtune` de MATLAB, tomando como punto de partida el modelo discreto obtenido mediante `tfest`, con este procedimiento se generaron parámetros iniciales del PID, que posteriormente fueron ajustados manualmente para obtener una respuesta más suave y con menor tiempo de establecimiento.

Dado que la implementación final debía ejecutarse en un microcontrolador Arduino, se adoptó una formulación de **PID incremental**, la cual es numéricamente robusta, reduce problemas de saturación integral y es menos sensible al ruido de medición. El controlador incremental se expresa como:

$$u(k) = u(k-1) + A\,e(k) + B\,e(k-1) + C\,e(k-2),$$

donde $e(k) = d_{\mathrm{ref}} - d_f(k)$ es el error en el instante k, y los coeficientes incrementales se relacionan con los parámetros clásicos del PID mediante discretización.

Los valores finales utilizados en la implementación fueron:

$$A_{\text{inc}} = 66.805, \qquad B_{\text{inc}} = -13.432, \qquad C_{\text{inc}} = 4.705.$$

Estos coeficientes provienen de escalar los valores obtenidos por MATLAB (222.6841, -44.7737, 15.6834) mediante un factor de 0.3, lo cual permitió adecuar las magnitudes de control a la dinámica real del carro y evitar saturación del actuador (PWM).

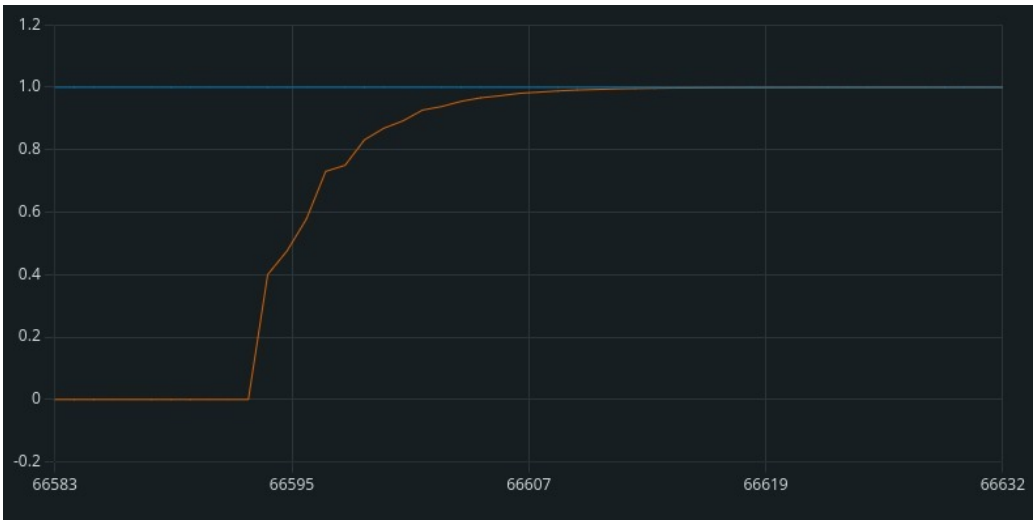


Figure 4: Respuesta al escalón obtenida en la simulación del PID implementado en Arduino.

La Fig. 4 muestra que el sistema converge suavemente hacia la referencia, sin sobrepaso significativo y con comportamiento estable, validando así el diseño del controlador.

5 Resultados

En esta sección se presentan los comportamientos experimentales del sistema sin control y con el controlador PID implementado, con el fin de comparar el desempeño dinámico en ambos casos.

5.1 Sistema sin compensación

Las pruebas sin controlador muestran que el vehículo no es capaz de mantener una distancia estable respecto al obstáculo. Al aplicar un PWM fijo, la distancia frontal disminuye de manera casi lineal, evidenciando que el sistema carece de mecanismos de regulación. Las respuestas de distancia para distintos niveles de PWM se muestran en la Fig. 5.

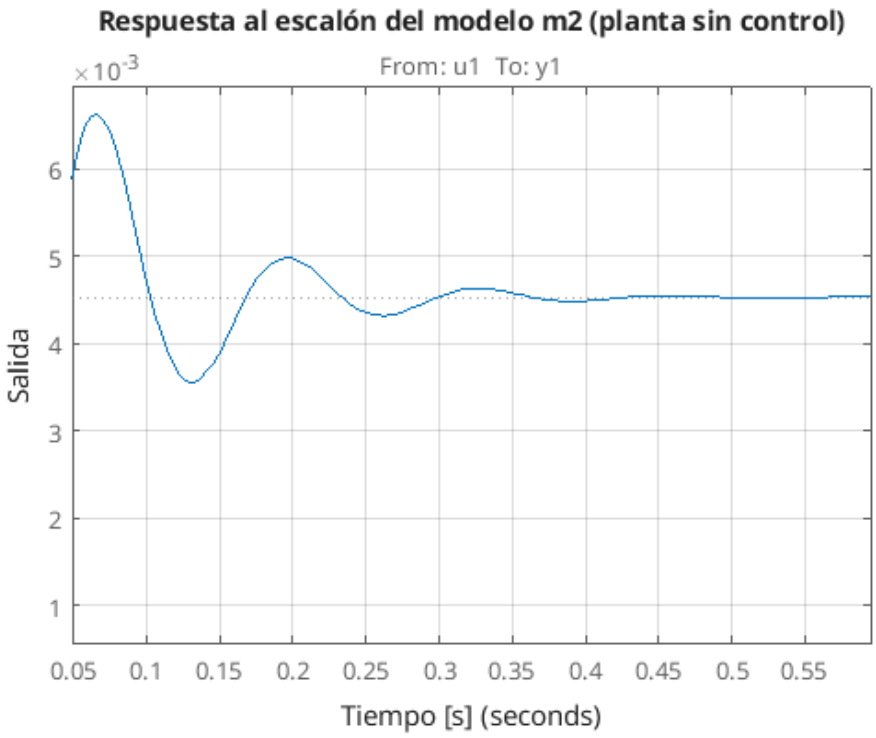


Figure 5: El sistema sin control no logra regular la distancia al obstáculo.

5.2 Sistema compensado con PID

Al implementar el controlador PID diseñado, el sistema presenta una respuesta estable y regulada frente a cambios en la distancia. La Fig. 6 muestra la respuesta al escalón obtenida con el PID, donde se observa una convergencia suave hacia la referencia y ausencia de sobrepaso significativo.

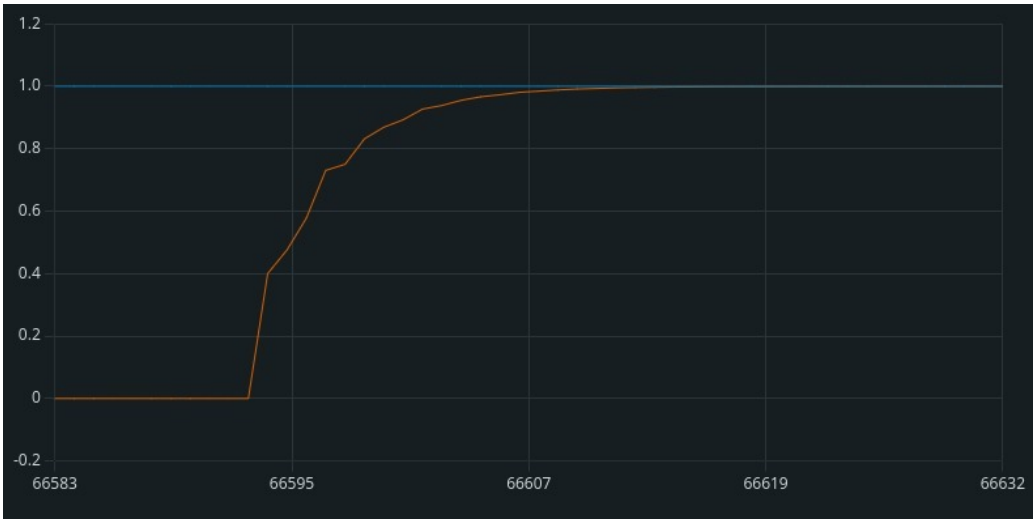


Figure 6: Respuesta de sistema compensado con PID. El controlador regula la distancia frontal de forma estable.

6 Análisis de Resultados

Los resultados obtenidos evidencian que el controlador PID diseñado cumple de manera efectiva con los objetivos de regulación planteados para el sistema, en las pruebas sin control, el carro se aproxima al obstáculo de forma no regulada, mostrando un comportamiento dominado únicamente por la velocidad de los motores y sin capacidad de mantener una distancia estable. Este comportamiento confirma que la planta, por sí sola, no posee mecanismos de estabilización ni rechazo de perturbaciones.

Una vez implementado el controlador PID, la dinámica del sistema cambia de forma significativa, la respuesta al escalón en lazo cerrado muestra una convergencia suave hacia la referencia, sin sobrepasos relevantes y con un error en estado estacionario prácticamente nulo. El controlador compensa adecuadamente las variaciones en la distancia detectada, ajustando la señal PWM para mantener el valor deseado incluso ante perturbaciones.

La comparación entre ambos escenarios demuestra que el PID logra transformar un sistema inherentemente no regulado en uno estable y capaz de mantener un comportamiento controlado. La dinámica obtenida en lazo cerrado es consistente con el modelo identificado, validando tanto el proceso de identificación como la metodología de diseño del controlador. Además, los resultados experimentales concuerdan con las simulaciones realizadas, lo cual confirma que los parámetros ajustados del PID son adecuados para la planta real.

En conjunto, los resultados permiten concluir que el control aplicado es preciso, robusto y cumple los objetivos propuestos en términos de regulación, estabilidad y rechazo de perturbaciones, garantizando un comportamiento seguro del carro durante el movimiento.

7 Conclusiones y Recomendaciones

7.1 Conclusiones

El proyecto cumplió de manera satisfactoria con los objetivos planteados, el modelo empírico obtenido fue adecuado para representar la dinámica principal del sistema y permitió diseñar un controlador PID funcional. Con el control implementado, el carro fue capaz de reaccionar correctamente ante un obstáculo y ajustar su distancia de manera rápida y estable, demostrando la efectividad del enfoque aplicado.

El trabajo permitió comprender de manera práctica el proceso completo de diseño de un sistema de control: desde la obtención de datos y modelado de la planta, hasta la implementación del PID en hardware real. Durante el desarrollo se identificó inicialmente una incompatibilidad entre el modelo empírico y el controlador diseñado, lo que llevó a un proceso de ajuste y validación que fortaleció la comprensión del comportamiento real de la planta y de los parámetros del PID.

El proyecto también permitió observar limitaciones prácticas del sistema, como la diferencia de respuesta entre los motores, que ocasionaba desviaciones laterales, este comportamiento mostró que, aunque el controlador regulaba la distancia frontal, el desempeño

en trayectoria dependía de factores mecánicos adicionales no incluidos en el modelo.

En conjunto, el sistema logró un control estable, preciso y funcional, cumpliendo con los objetivos académicos y prácticos establecidos.

7.2 Recomendaciones

Para trabajos futuros, se recomienda explorar controladores más avanzados que permitan no solo regular la distancia frontal, sino también mejorar el seguimiento de trayectoria y compensar diferencias entre motores, entre las posibles mejoras se destacan:

- implementar controladores independientes por rueda, integrando control diferencial;
- añadir filtrado a la señal del sensor ultrasónico para reducir ruido y mejorar la medición;
- incorporar sensores adicionales como encoders o unidades inerciales (IMU);
- utilizar motores con mayor precisión o realimentación interna;
- extender el control para permitir giros, evasión lateral y navegación autónoma.

Además, se sugiere continuar refinando el proceso de identificación de la planta, considerando modelos más completos o técnicas avanzadas de identificación no lineal.

8 Bibliografía

[1] C. Rincón, J. Martínez, y A. Rodríguez, "Diseño sistema PID para el control de distancia de un vehículo seguidor de objetos," *19th LACCEI Multi-Conference*, Buenos Aires, Argentina, 2021. :contentReference[oaicite:0]index=0

[2] J. I. Collazo, E. Gorrostieta, J. Pedraza, U. Villaseñor, R. Romero, y M. González, "Modelación de un robot móvil de dos ruedas con tracción diferencial," *8º Congreso Nacional de Mecatrónica*, Veracruz, México, 2009. :contentReference[oaicite:1]index=1

[3] ElecFreaks, "HC-SR04 Ultrasonic Sensor Datasheet," 2013. [En línea]. Disponible en: `https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf`

[4] STMicroelectronics, "L298N Dual H-Bridge Motor Driver Datasheet," 2010. [En línea]. Disponible en: `https://www.st.com/resource/en/datasheet/l298.pdf`

[5] Arduino Documentation, "PWM — Pulse Width Modulation," 2024. [En línea]. Disponible en: `https://docs.arduino.cc/learn/microcontrollers/analog-output`

[6] MathWorks, "System Identification Toolbox Documentation," 2024. [En línea]. Disponible en: https://www.mathworks.com/help/ident

[7] MathWorks, "PID Tuning and Control Design," 2024. [En línea]. Disponible en: https://www.mathworks.com/help/control/ref/pidtuner-app.html

[8] K. Ogata, *Ingeniería de Control Moderna*, 5ta ed., Pearson, 2010.

A Código Fuente

Listing 1: Control PID para la planta

```

#include <NewPing.h>
#include <Servo.h>

Servo servoSensor;

#define TRIG_PIN A3
#define ECHO_PIN A2
#define MAX_DISTANCE 200
NewPing sonar(TRIG_PIN, ECHO_PIN, MAX_DISTANCE);

const int ENA = 5;
const int IN1 = 10;
const int IN2 = 8;
const int ENB = 6;
const int IN3 = 11;
const int IN4 = 7;

const double Ts = 0.05;
unsigned long t0;

double ref = 0.30;

const double Ainc = 0.25 * 222.6841;
const double Binc = 0.25 * -44.7737;
const double Cinc = 0.25 * 15.6834;

double ek   = 0.0;
double ekm1 = 0.0;
double ekm2 = 0.0;
double uk   = 0.0;
double ukm1 = 0.0;

double d_ema = 0.0;
const double EMA_ALPHA = 0.2;

const double CTRL_GAIN = 13.0;
```

```
const int     PWM_MAX    = 220;
const int     PWM_MIN    = 70;

const double REF_BAND = 0.02;

const double D_EMERGENCY = 0.15;

bool INVERT_DIR = true;

const int OFFSET_RIGHT_FWD = 21;
const int OFFSET_RIGHT_REV = 20;

const int SERVO_CENTER = 90;
const int SERVO_LEFT1  = 120;
const int SERVO_LEFT2  = 140;
const int SERVO_RIGHT1 = 60;
const int SERVO_RIGHT2 = 40;

const int TURN_PWM     = 130;
const int TURN_STEPS   = 12;

enum Mode { MODE_FOLLOW, MODE_AVOID_TURN };
Mode mode = MODE_FOLLOW;
int  turnDir   = 0;
int  turnSteps = 0;

static inline float median3(float a, float b, float c) {
  if (a > b) { float t = a; a = b; b = t; }
  if (b > c) { float t = b; b = c; c = t; }
  if (a > b) { float t = a; a = b; b = t; }
  return b;
}

double readDistanceM() {
  unsigned int c1 = sonar.ping_cm(); if (!c1) c1 = MAX_DISTANCE;
  unsigned int c2 = sonar.ping_cm(); if (!c2) c2 = MAX_DISTANCE;
  unsigned int c3 = sonar.ping_cm(); if (!c3) c3 = MAX_DISTANCE;
  float cm = median3(c1, c2, c3);
  return cm / 100.0;
}

double meanDistanceAtAngle(int angleDeg, int nSamples = 3) {
  servoSensor.write(angleDeg);
  delay(150);
  double sum = 0.0;
  for (int i = 0; i < nSamples; i++) {
    double d = readDistanceM();
    if (d > 2.0) d = 2.0;
    sum += d;
    delay(40);
  }
```

```
  return sum / (double)nSamples;
}

double scanSideMean(int side) {
  double m1, m2;
  if (side < 0) {
    m1 = meanDistanceAtAngle(SERVO_LEFT1);
    m2 = meanDistanceAtAngle(SERVO_LEFT2);
  } else {
    m1 = meanDistanceAtAngle(SERVO_RIGHT1);
    m2 = meanDistanceAtAngle(SERVO_RIGHT2);
  }
  return 0.5 * (m1 + m2);
}

void driveSignedPWM(int uSigned) {
  int s = (uSigned >= 0) ? +1 : -1;
  if (INVERT_DIR) s = -s;

  int base = abs(uSigned);
  if (base > PWM_MAX) base = PWM_MAX;

  int valLeft  = base;
  int valRight = base;

  if (s > 0) {
    valRight -= OFFSET_RIGHT_FWD;
    if (valRight < 0) valRight = 0;
  } else {
    valRight -= OFFSET_RIGHT_REV;
    if (valRight < 0) valRight = 0;
  }

  digitalWrite(IN1, (s > 0) ? HIGH : LOW);
  digitalWrite(IN2, (s > 0) ? LOW  : HIGH);
  digitalWrite(IN3, (s > 0) ? HIGH : LOW);
  digitalWrite(IN4, (s > 0) ? LOW  : HIGH);

  analogWrite(ENA, valLeft);
  analogWrite(ENB, valRight);
}

void driveTurnInPlace(int dir, int pwm) {
  if (pwm > PWM_MAX) pwm = PWM_MAX;
  if (pwm < PWM_MIN) pwm = PWM_MIN;

  if (dir > 0) {
    digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW);  digitalWrite(IN4, HIGH);
  } else {
    digitalWrite(IN1, LOW);  digitalWrite(IN2, HIGH);
```

```
    digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
  }

  analogWrite(ENA, pwm);
  analogWrite(ENB, pwm);
}

void resetPID() {
  uk   = 0.0;
  ukm1 = 0.0;
  ekm1 = 0.0;
  ekm2 = 0.0;
}

void startAvoidance() {
  driveSignedPWM(0);
  resetPID();

  double dLeftMean  = scanSideMean(-1);
  double dRightMean = scanSideMean(+1);

  servoSensor.write(SERVO_CENTER);
  delay(150);

  if (dLeftMean > dRightMean) turnDir = -1;
  else                        turnDir = +1;

  turnSteps = TURN_STEPS;
  mode = MODE_AVOID_TURN;

  t0 = millis();
}

void setup() {
  Serial.begin(115200);

  servoSensor.attach(9);
  servoSensor.write(SERVO_CENTER);
  delay(200);

  pinMode(ENA, OUTPUT); pinMode(ENB, OUTPUT);
  pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);

  digitalWrite(IN1, LOW);  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);  digitalWrite(IN4, LOW);

  d_ema = readDistanceM();
  t0 = millis();
}

```

```
void loop() {
  if (millis() - t0 < (unsigned long)(Ts * 1000.0)) return;
  t0 += (unsigned long)(Ts * 1000.0);

  double dist = readDistanceM();
  if (dist > 2.0) dist = 2.0;

  d_ema = (1.0 - EMA_ALPHA) * d_ema + EMA_ALPHA * dist;

  if (mode == MODE_FOLLOW) {
    ek = ref - d_ema;

    if (d_ema < D_EMERGENCY) {
      startAvoidance();
      Serial.print(ref, 3);
      Serial.print(" ");
      Serial.println(d_ema, 3);
      return;
    }

    if (fabs(ek) <= REF_BAND) {
      startAvoidance();
      Serial.print(ref, 3);
      Serial.print(" ");
      Serial.println(d_ema, 3);
      return;
    }

    uk = ukm1 + Ainc * ek + Binc * ekm1 + Cinc * ekm2;

    if (uk >  10.0) uk =  10.0;
    if (uk < -10.0) uk = -10.0;

    double uAbs = fabs(uk) * CTRL_GAIN;
    int pwm = (int)uAbs;
    if (pwm > PWM_MAX) pwm = PWM_MAX;
    if (pwm < PWM_MIN) pwm = PWM_MIN;

    int sign = (uk >= 0.0) ? +1 : -1;
    int uSigned = sign * pwm;

    driveSignedPWM(uSigned);

    ekm2 = ekm1;
    ekm1 = ek;
    ukm1 = uk;

    Serial.print(ref, 3);
    Serial.print(" ");
    Serial.println(d_ema, 3);
  }
```

```
  else if (mode == MODE_AVOID_TURN) {
    if (turnSteps > 0) {
      driveTurnInPlace(turnDir, TURN_PWM);
      turnSteps--;
    } else {
      driveSignedPWM(0);
      resetPID();
      mode = MODE_FOLLOW;
      servoSensor.write(SERVO_CENTER);
      delay(100);
    }

    Serial.print(ref, 3);
    Serial.print(" ");
    Serial.println(d_ema, 3);
  }
}
```

Listing 2: Control en modo evasor de obstáculos mediante PID incremental

```
#include <NewPing.h>
#include <Servo.h>

// ----------------------------
// Configuraci n de hardware
// ----------------------------
Servo servoSensor;

#define TRIG_PIN     A3
#define ECHO_PIN     A2
#define MAX_DISTANCE 200

NewPing sonar(TRIG_PIN, ECHO_PIN, MAX_DISTANCE);

const int ENA = 5;
const int IN1 = 10;
const int IN2 = 8;
const int ENB = 6;
const int IN3 = 11;
const int IN4 = 7;

// ----------------------------
// Par metros de control
// ----------------------------
const double Ts = 0.05;   // Per odo de muestreo [s]
unsigned long t0;

double ref = 0.30;        // Referencia de distancia [m]

// Coeficientes PID incremental (discretizado)
const double Ainc = 0.25 * 222.6841;
```

```
const double Binc = 0.25 * -44.7737;
const double Cinc = 0.25 * 15.6834;

double ek   = 0.0;
double ekm1 = 0.0;
double ekm2 = 0.0;
double uk   = 0.0;
double ukm1 = 0.0;

// Filtro EMA para la medici n de distancia
double d_ema = 0.0;
const double EMA_ALPHA = 0.2;

// Ganancias y l mites del actuador
const double CTRL_GAIN = 13.0;
const int    PWM_MAX   = 220;
const int    PWM_MIN   = 70;

// Banda muerta alrededor de la referencia
const double REF_BAND = 0.02;

// Distancia de emergencia para evasi n
const double D_EMERGENCY = 0.15;

// Ajustes direccionales
bool INVERT_DIR = true;

const int OFFSET_RIGHT_FWD = 21;
const int OFFSET_RIGHT_REV = 20;

//  ngulos  del servo
const int SERVO_CENTER = 90;
const int SERVO_LEFT1  = 120;
const int SERVO_LEFT2  = 140;
const int SERVO_RIGHT1 = 60;
const int SERVO_RIGHT2 = 40;

// Par metros de giro evasivo
const int TURN_PWM   = 130;
const int TURN_STEPS = 12;

// ----------------------------
// Modos de operaci n
// ----------------------------
enum Mode { MODE_FOLLOW, MODE_AVOID_TURN };
Mode mode = MODE_FOLLOW;

int turnDir   = 0;
int turnSteps = 0;

// ----------------------------
```

```
// Utilitarios
// ----------------------------
static inline float median3(float a, float b, float c) {
  if (a > b) { float t = a; a = b; b = t; }
  if (b > c) { float t = b; b = c; c = t; }
  if (a > b) { float t = a; a = b; b = t; }
  return b;
}

double readDistanceM() {
  unsigned int c1 = sonar.ping_cm(); if (!c1) c1 = MAX_DISTANCE;
  unsigned int c2 = sonar.ping_cm(); if (!c2) c2 = MAX_DISTANCE;
  unsigned int c3 = sonar.ping_cm(); if (!c3) c3 = MAX_DISTANCE;

  float cm = median3(c1, c2, c3);
  return cm / 100.0;  // Pasar a metros
}

double meanDistanceAtAngle(int angleDeg, int nSamples = 3) {
  servoSensor.write(angleDeg);
  delay(150);

  double sum = 0.0;
  for (int i = 0; i < nSamples; i++) {
    double d = readDistanceM();
    if (d > 2.0) d = 2.0;  // Limitar a 2 m
    sum += d;
    delay(40);
  }
  return sum / (double)nSamples;
}

double scanSideMean(int side) {
  double m1, m2;
  if (side < 0) {
    m1 = meanDistanceAtAngle(SERVO_LEFT1);
    m2 = meanDistanceAtAngle(SERVO_LEFT2);
  } else {
    m1 = meanDistanceAtAngle(SERVO_RIGHT1);
    m2 = meanDistanceAtAngle(SERVO_RIGHT2);
  }
  return 0.5 * (m1 + m2);
}

// ----------------------------
// Actuaci n sobre motores
// ----------------------------
void driveSignedPWM(int uSigned) {
  int s = (uSigned >= 0) ? +1 : -1;
  if (INVERT_DIR) s = -s;

```

```
  int base = abs(uSigned);
  if (base > PWM_MAX) base = PWM_MAX;

  int valLeft  = base;
  int valRight = base;

  if (s > 0) {
    valRight -= OFFSET_RIGHT_FWD;
    if (valRight < 0) valRight = 0;
  } else {
    valRight -= OFFSET_RIGHT_REV;
    if (valRight < 0) valRight = 0;
  }

  digitalWrite(IN1, (s > 0) ? HIGH : LOW);
  digitalWrite(IN2, (s > 0) ? LOW  : HIGH);
  digitalWrite(IN3, (s > 0) ? HIGH : LOW);
  digitalWrite(IN4, (s > 0) ? LOW  : HIGH);

  analogWrite(ENA, valLeft);
  analogWrite(ENB, valRight);
}

void driveTurnInPlace(int dir, int pwm) {
  if (pwm > PWM_MAX) pwm = PWM_MAX;
  if (pwm < PWM_MIN) pwm = PWM_MIN;

  if (dir > 0) {
    digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW);  digitalWrite(IN4, HIGH);
  } else {
    digitalWrite(IN1, LOW);  digitalWrite(IN2, HIGH);
    digitalWrite(IN3, HIGH); digitalWrite(IN4, LOW);
  }

  analogWrite(ENA, pwm);
  analogWrite(ENB, pwm);
}

// ----------------------------
// Gesti n del PID
// ----------------------------
void resetPID() {
  uk   = 0.0;
  ukm1 = 0.0;
  ekm1 = 0.0;
  ekm2 = 0.0;
}

// ----------------------------
// L gica de evasi n
```

```
// ----------------------------
void startAvoidance() {
  driveSignedPWM(0);
  resetPID();

  double dLeftMean  = scanSideMean(-1);
  double dRightMean = scanSideMean(+1);

  servoSensor.write(SERVO_CENTER);
  delay(150);

  if (dLeftMean > dRightMean) turnDir = -1;
  else                        turnDir = +1;

  turnSteps = TURN_STEPS;
  mode      = MODE_AVOID_TURN;

  t0 = millis();
}

// ----------------------------
// Setup y bucle principal
// ----------------------------
void setup() {
  Serial.begin(115200);

  servoSensor.attach(9);
  servoSensor.write(SERVO_CENTER);
  delay(200);

  pinMode(ENA, OUTPUT); pinMode(ENB, OUTPUT);
  pinMode(IN1, OUTPUT); pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT); pinMode(IN4, OUTPUT);

  digitalWrite(IN1, LOW);  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);  digitalWrite(IN4, LOW);

  d_ema = readDistanceM();
  t0    = millis();
}

void loop() {
  if (millis() - t0 < (unsigned long)(Ts * 1000.0)) return;
  t0 += (unsigned long)(Ts * 1000.0);

  double dist = readDistanceM();
  if (dist > 2.0) dist = 2.0;

  // Filtro EMA
  d_ema = (1.0 - EMA_ALPHA) * d_ema + EMA_ALPHA * dist;

```

```
  if (mode == MODE_FOLLOW) {
    ek = ref - d_ema;

    // Condiciones para iniciar evasi n
    if (d_ema < D_EMERGENCY) {
      startAvoidance();
      Serial.print(ref, 3);
      Serial.print(" ");
      Serial.println(d_ema, 3);
      return;
    }

    if (fabs(ek) <= REF_BAND) {
      startAvoidance();
      Serial.print(ref, 3);
      Serial.print(" ");
      Serial.println(d_ema, 3);
      return;
    }

    // PID incremental
    uk = ukm1 + Ainc * ek + Binc * ekm1 + Cinc * ekm2;

    if (uk >  10.0) uk =  10.0;
    if (uk < -10.0) uk = -10.0;

    double uAbs = fabs(uk) * CTRL_GAIN;
    int pwm = (int)uAbs;
    if (pwm > PWM_MAX) pwm = PWM_MAX;
    if (pwm < PWM_MIN) pwm = PWM_MIN;

    int sign    = (uk >= 0.0) ? +1 : -1;
    int uSigned = sign * pwm;

    driveSignedPWM(uSigned);

    ekm2 = ekm1;
    ekm1 = ek;
    ukm1 = uk;

    Serial.print(ref, 3);
    Serial.print(" ");
    Serial.println(d_ema, 3);
  }
  else if (mode == MODE_AVOID_TURN) {
    if (turnSteps > 0) {
      driveTurnInPlace(turnDir, TURN_PWM);
      turnSteps--;
    } else {
      driveSignedPWM(0);
      resetPID();
```

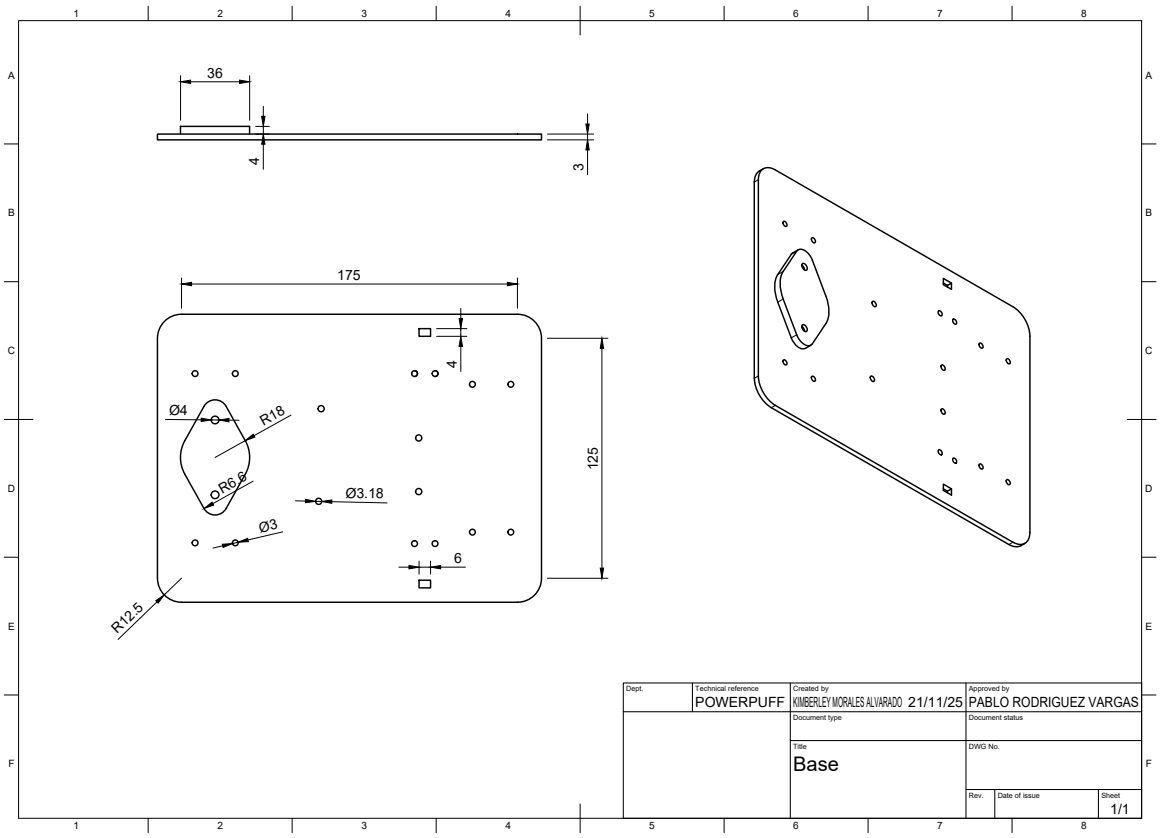
```
      mode = MODE_FOLLOW;
      servoSensor.write(SERVO_CENTER);
      delay(100);
    }

    Serial.print(ref, 3);
    Serial.print(" ");
    Serial.println(d_ema, 3);
  }
}
```

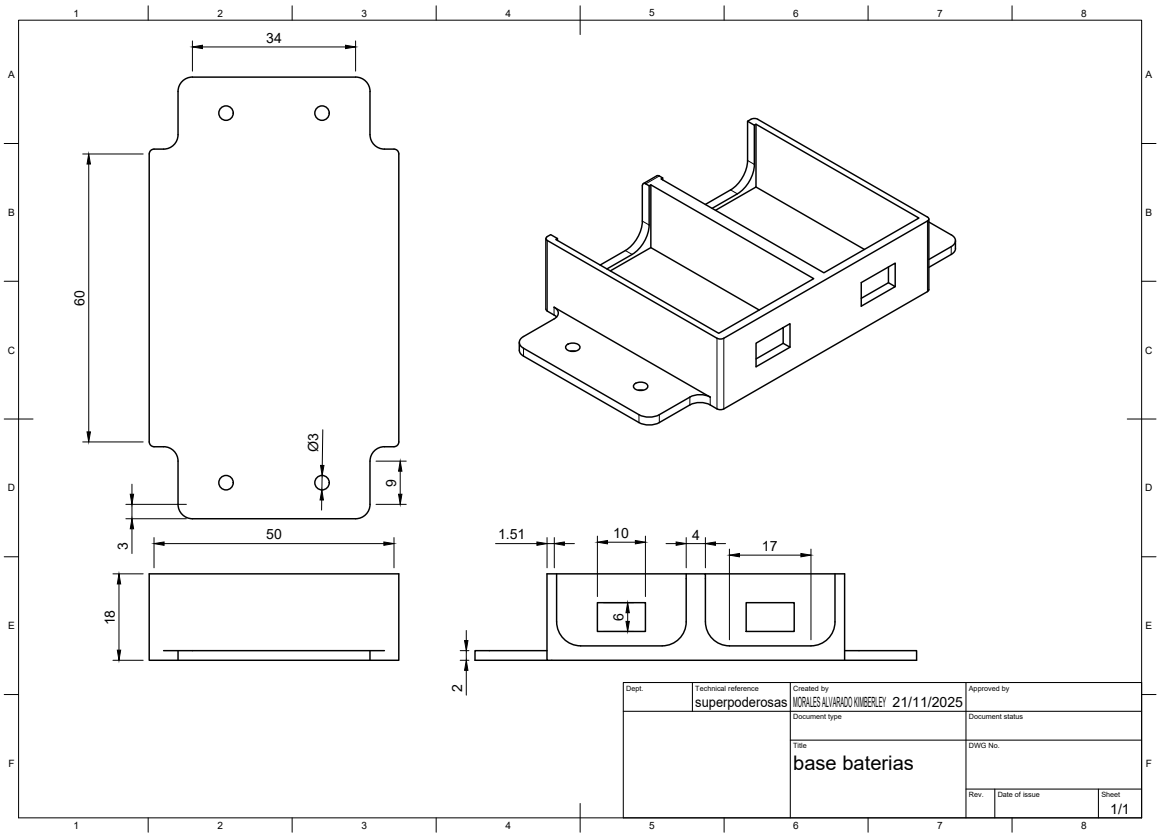
B Planos de Construcción

A continuación se presentan los planos de fabricación utilizados para la construcción del prototipo del carro evasor de obstáculos, cada plano fue modelado en CAD e impreso en 3D para conformar la estructura completa del robot.

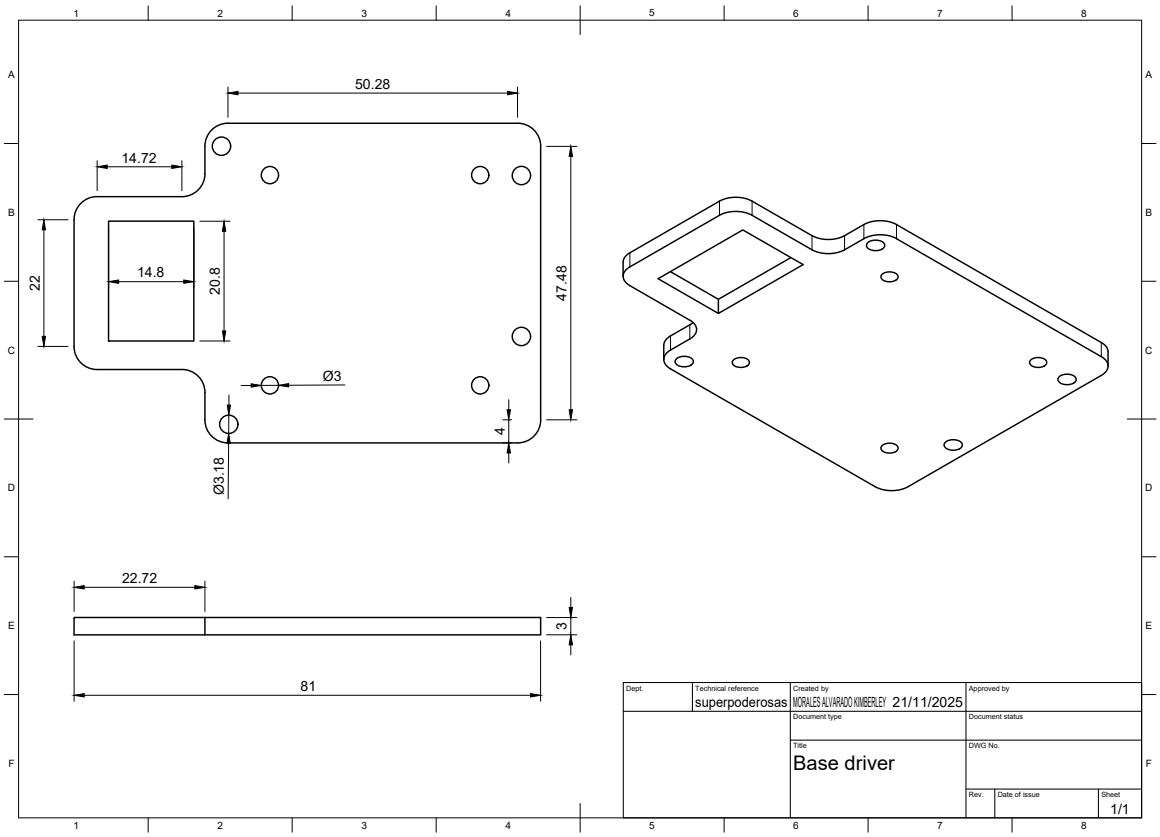
Plano 1: Base Principal



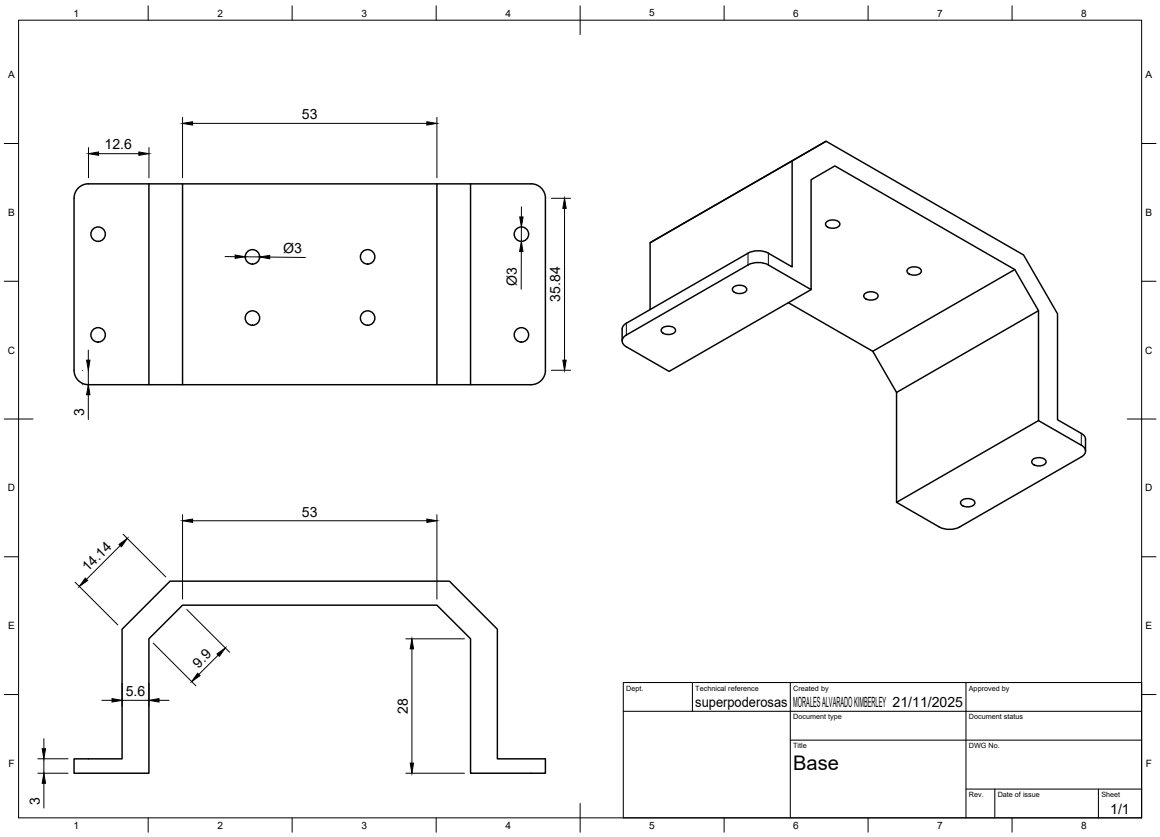
Plano 2: Base de Baterías



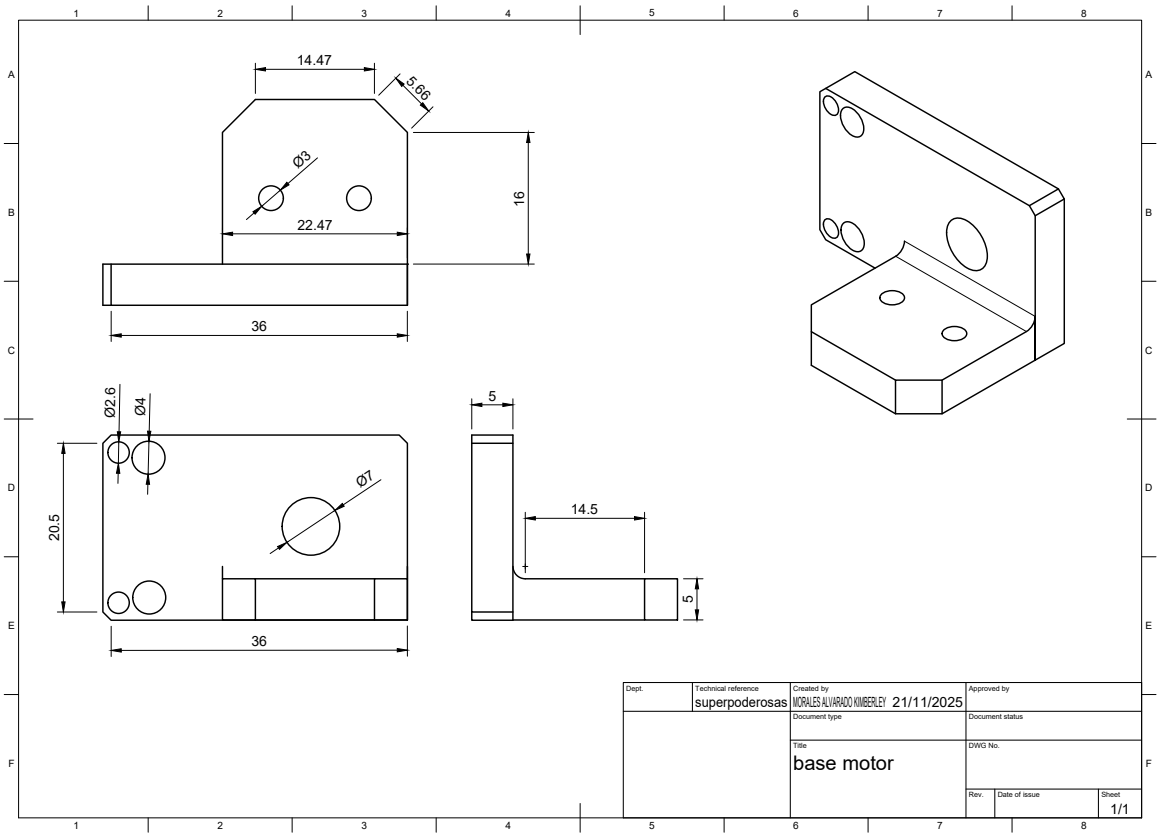
Plano 3: Base del Driver L298N



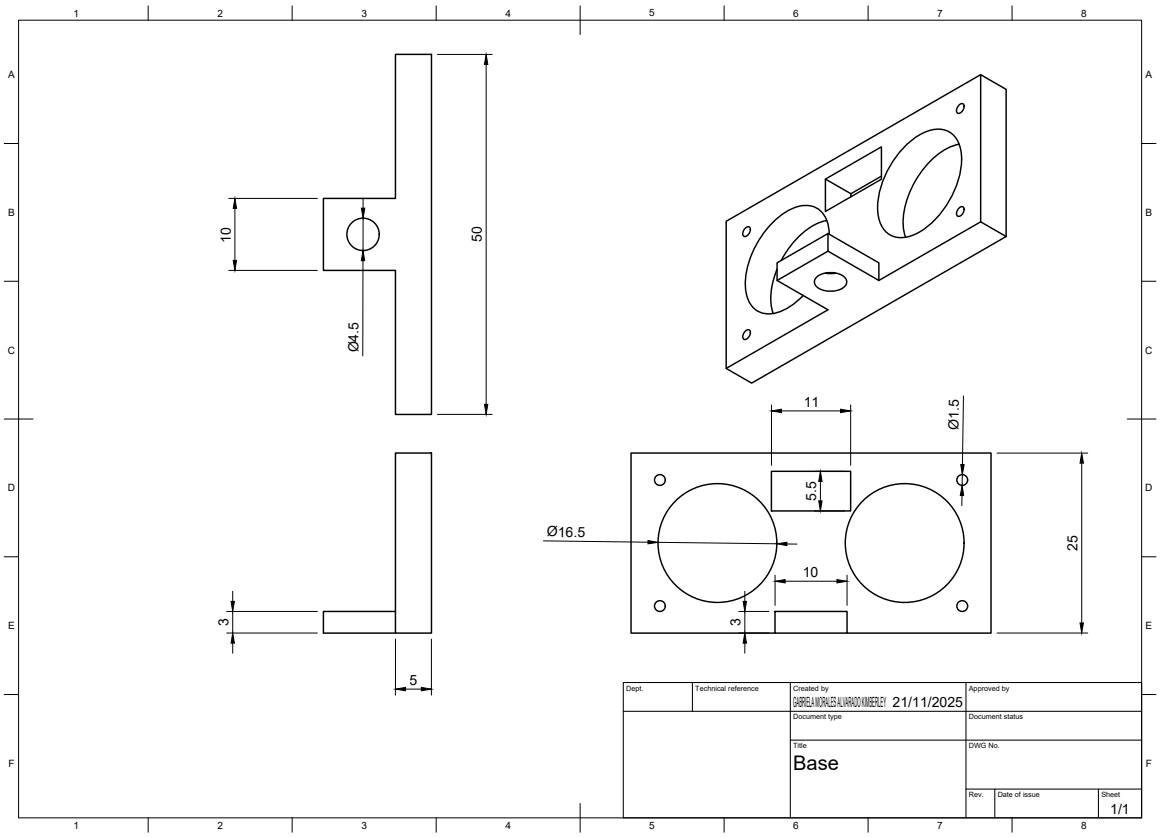
Plano 4: Base Frontal



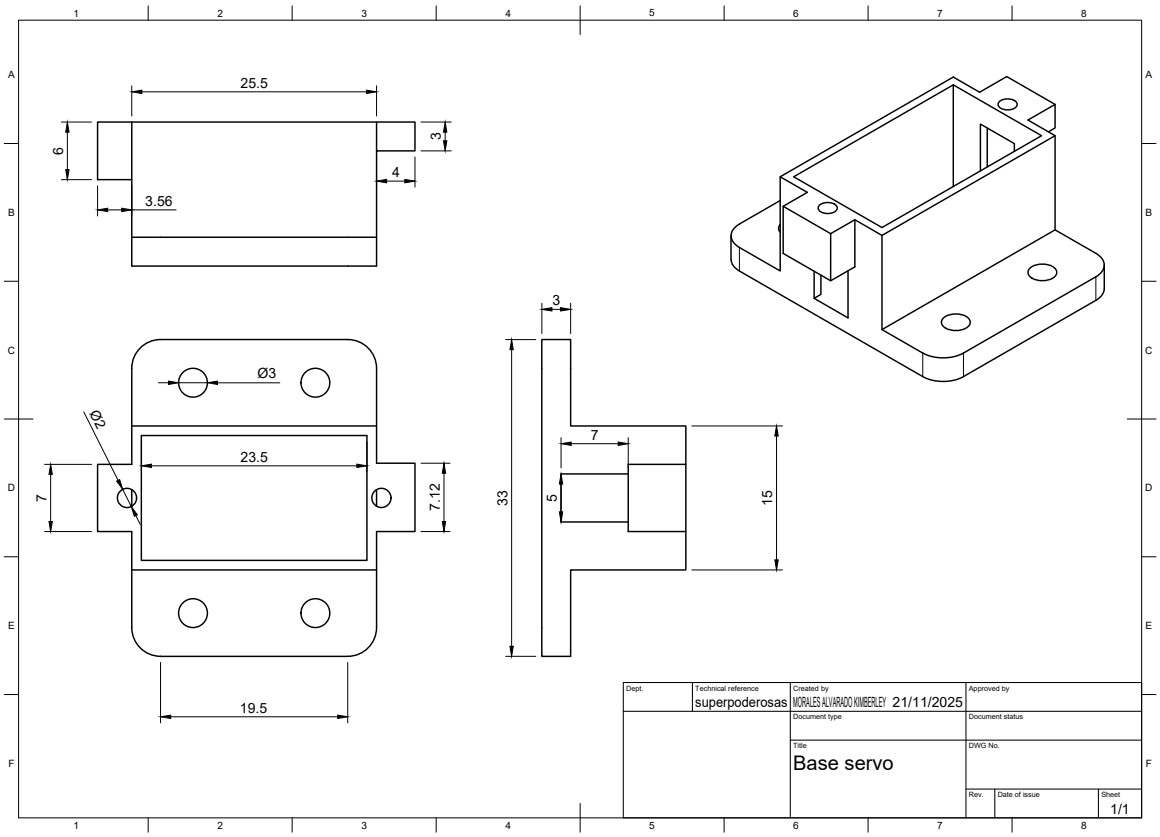
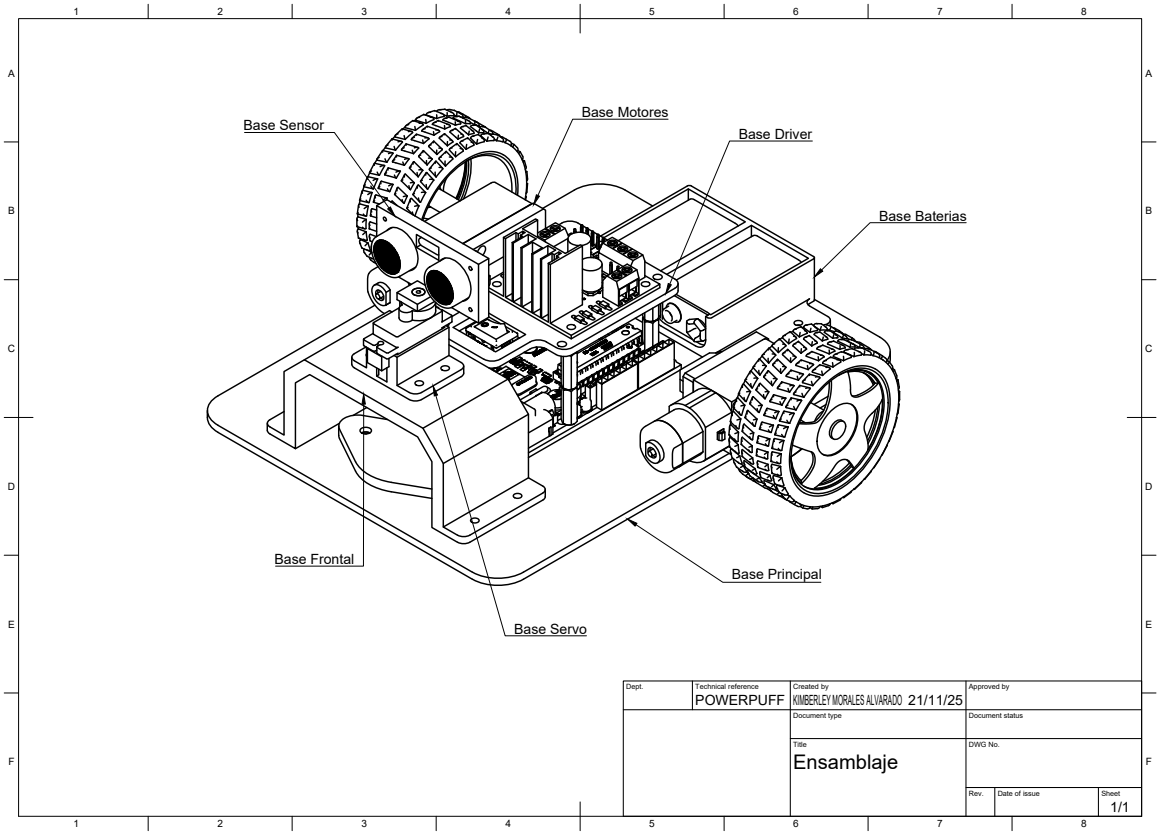
Plano 5: Base de Motores



Plano 6: Base del Sensor Ultrasonido



Plano 7: Base del Servomotor



Plano 8: Ensamblaje General